

## CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 06 · LECTURE NOTES

# Key Exchange and Public Keys

*Establishing a shared secret with a stranger.*

## MODULE OUTCOMES

- Explain Diffie–Hellman and what it does and does not provide.
- Explain the hardness assumptions and their relationship.
- Explain why public-key encryption is rarely used directly.
- Explain forward secrecy and why ephemeral keys matter.
- Choose between the curve and finite-field settings.

## 1 Diffie–Hellman

$A \rightarrow B: g^a$        $B \rightarrow A: g^b$       shared secret:  $g^{ab}$

- $g$  — a public generator of a group in which the discrete logarithm problem is believed hard. Public, fixed, and shared by everyone.
- $a, b$  — each party's private value, generated fresh from a CSPRNG (Module 02) and **never transmitted**.
- $g^{ab}$  — computed as  $(g^b)^a$  by A and as  $(g^a)^b$  by B. **The same value, from different inputs, which is the whole trick.**
- **The adversary** — sees  $g, g^a$ , and  $g^b$ , and must compute  $g^{ab}$ , which is the computational Diffie–Hellman problem and is believed hard (§2). **Write this out by hand once with small numbers** — it is the most elegant result in the subject and it takes five minutes.

**DIFFIE–HELLMAN GIVES YOU A SHARED SECRET WITH SOMEBODY**

**Unauthenticated, it is wholly vulnerable to an active adversary. A machine in the middle runs the exchange separately with each party and relays the traffic between them** — producing two perfectly secure channels, both terminating at the adversary, who reads and may modify everything.

**And neither party can detect this from the exchange itself**, because every message is correctly formed, the mathematics is valid, and both ends successfully derive a key that their peer also holds. **The cryptography worked exactly as specified.**

**So key exchange must be authenticated** — by a signature over the exchange transcript (Module 07), by a pre-shared key, or by a password-authenticated key exchange where a low-entropy shared secret suffices without being brute-forceable.

**Which is why Module 07 and the whole certificate problem exist: the hard part was never agreeing on a key — it was knowing who you agreed with**, and that turns out not to be a cryptographic problem at all (Module 01 §2's third limitation).

## 2 The assumptions

| Problem                      | The statement                                    | What depends on it                                                                                                             |
|------------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Discrete logarithm           | Given $g$ and $g^a$ , find $a$ .                 | Diffie–Hellman, DSA, ECDSA, Ed25519.                                                                                           |
| Computational Diffie–Hellman | Given $g^a$ and $g^b$ , compute $g^{ab}$ .       | DH key exchange. A potentially weaker assumption than discrete log — breaking DL breaks this, but not necessarily the reverse. |
| Integer factoring            | Given $n = pq$ for large primes, find $p$ .      | RSA, for both signing and encryption.                                                                                          |
| Short vectors in lattices    | Find a short non-zero vector in a given lattice. | The post-quantum schemes (Module 12), which is why they rest on a different assumption rather than a better algorithm.         |

**THE SECURITY RESTS ON UNPROVED ASSUMPTIONS, AND THAT IS THE HONEST POSITION**

**Nobody has proved that factoring or discrete log is hard. Even  $P \neq NP$  would not settle the question,** since neither problem is known to be NP-hard — both sit in the suspected gap between P and NP-complete (CSCE 637 Module 09), which is one reason that gap is interesting.

**What we actually have is several decades of concentrated attention by capable people without a polynomial-time algorithm** — which is substantial evidence and is categorically not proof, and the distinction is worth keeping clear because the whole edifice rests on it.

**And the assumptions are not independent of the computational model. A sufficiently large quantum computer breaks both factoring and discrete log in polynomial time** by Shor's algorithm — so the assumption was always 'hard for a classical adversary', which Module 12 addresses.

**So the honest claim is conditional: 'secure assuming the discrete logarithm problem is hard in this group against a classical adversary' — and Module 13 is about stating that condition rather than quietly dropping it,** which is the program's closing rule in this course's vocabulary.

### 3 Public-key encryption, and hybrid construction

- **Public-key operations are orders of magnitude slower** than symmetric ones — a factor of hundreds to thousands — and the gap is large enough to dominate at any meaningful scale.
- **And they encrypt only small messages. RSA can encrypt a plaintext smaller than its modulus, less padding**, so a document does not fit at all and must be split or, correctly, handled differently.
- **So the universal pattern is hybrid: use public-key cryptography to establish or transport a symmetric key, then encrypt the actual data symmetrically** with an AEAD construction (Module 04).
- **Which is what TLS does** (Module 08), **what encrypted email and messaging do**, and **what envelope encryption does** (Module 10 §2) — the pattern is so universal that the libraries expose it directly as a sealed-box or key-encapsulation operation.
- **And raw RSA encryption is a hazard in its own right. Textbook RSA is deterministic — so identical plaintexts give identical ciphertexts — and malleable, so an adversary can transform ciphertexts predictably. Padding (OAEP) is mandatory, and getting the padding validation wrong produced Bleichenbacher's attack and its many descendants. If you are calling an RSA encrypt function directly, stop — the correct operation is a key encapsulation or a sealed box.**

### 4 Forward secrecy

#### FORWARD SECRECY MEANS A KEY COMPROMISED TOMORROW DOES NOT DECRYPT TODAY'S TRAFFIC

If a session key is derived from a long-term private key, then compromising that key at any future point decrypts every past session that was recorded — and that is a realistic adversary model, because storing traffic is cheap and an adversary may obtain the key years later by compromise, by legal process, or by the key outliving its protection.

**So use an ephemeral exchange: generate a fresh Diffie–Hellman keypair for every session, authenticate it using the long-term key, derive the session key from the ephemeral exchange, and then discard the ephemeral private value.**

**The long-term key then only ever signs and never decrypts** — so its later compromise permits impersonation going forward, which is serious, and reveals nothing about past sessions at all, which is the property being bought.

**Which is why TLS 1.3 removed non-ephemeral key exchange entirely** (Module 08) rather than merely deprecating it — **a standards body deleting an option so that it cannot be selected**, which is this course's thesis appearing in standards form and is worth noticing as a design pattern.

- **Use X25519 for key exchange. Fast, 32-byte keys, no parameter choices to make, and no invalid-curve hazard — every 32-byte string is a valid input**, which eliminates a whole validation step and the bugs in it.
- **Elliptic curves provide equivalent security at much smaller key sizes** — a 256-bit curve against roughly a 3072-bit finite field — which is why they displaced the finite-field setting for new deployments.

- **Avoid implementing curve arithmetic yourself.** Point validation, handling of the identity element, cofactor subtleties, and non-constant-time scalar multiplication have each produced real, exploited breaks — this is among the least forgiving code in the field.
- **And avoid custom curves and custom groups** — parameter selection is exactly where the subtle weaknesses hide, and a group with a small subgroup or a special structure can be catastrophically weak while looking entirely ordinary.
- **Which is Module 01 §4 once more**, arriving in the place where it has the strongest historical support. X25519 was specifically designed so that misuse is difficult — no invalid points, no exposed cofactor subtleties, no parameters — and that design goal is the reason it is the recommendation, rather than its performance.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                   | What it covers for this module                                                                          |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Boneh & Shoup, chapters 10 through 12 (free)               | §1 through §3 rigorously — the assumptions, their relationships, and what the reductions establish.     |
| Aumasson — Serious Cryptography, chapters 10 through 12    | RSA, Diffie–Hellman, and elliptic curves at engineering level, with the practical hazards foregrounded. |
| Bernstein — Curve25519, and the SafeCurves criteria (free) | §4's design argument — what makes a curve hard to misuse, stated as explicit criteria.                  |
| Diffie & Hellman — New Directions in Cryptography (free)   | §1 in the original, from 1976 — short, readable, and one of the genuinely pivotal papers in computing.  |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Perform a Diffie–Hellman exchange by hand** with small numbers, both sides.
- 2 **Verify both parties compute the same value** and that you cannot get it from the public values alone.
- 3 **Write out the machine-in-the-middle attack** on an unauthenticated exchange.
- 4 **Explain why neither party can detect it** from the exchange.
- 5 **State the four hardness assumptions** and what depends on each.
- 6 **Explain why  $P \neq NP$  would not settle the security of RSA.**
- 7 **Measure RSA versus AES throughput** on the same machine.

- 8 **Implement the hybrid pattern** with a sealed-box API.
- 9 **Determine whether a TLS connection you use has forward secrecy**, and how you can tell.
- 10 **Write the conditional security claim** for your own system's key exchange, with the assumption named.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the Diffie–Hellman exchange and say what each party computes.
- 2 What does unauthenticated DH actually give you?
- 3 Why can the machine-in-the-middle not be detected from the exchange?
- 4 Name four hardness assumptions and what rests on each.
- 5 Why is CDH possibly weaker than discrete log?
- 6 Why would  $P \neq NP$  not settle these questions?
- 7 Why is everything real hybrid, and give two reasons.
- 8 Why is raw RSA encryption a hazard?
- 9 Define forward secrecy and say what it requires.
- 10 Why use X25519, and what was its design goal?